Sclera ADC AI Server

AI Features Documentation

Intelligent Asset Data Collection platform for automated identification, classification, and validation of building and industrial equipment assets

April 2026 | Port 8002 | FastAPI + Docker

GPT-4o / Grok-3 / Google Cloud Vision / SerpAPI / OpenAI Embeddings

AI Capability Summary

Capability	AI Technology
Equipment image recognition	GPT-4o / GPT-4o-mini (Vision)
OCR nameplate reading	GPT-4o / Google Cloud Vision
Asset description generation	Grok-3 / Grok-4 (xAI)
Semantic classification search	OpenAI text-embedding-3-large
Web-based asset lookup	SerpAPI (Google Search + Google Lens)
Column / field mapping	GPT-4o
Specification validation	GPT-4o
Field type inference	Grok API

Table of Contents

1 Project Overview

2 AI Features

2.1 Asset Image Classification (Mobile)

2.2 Asset Classification from Metadata (Mobile)

2.3 Asset Classification from Image (Mobile)

2.4 JACS Code Prediction from Metadata (Mobile)

2.5 Specification Validation

2.6 Column / Field Mapping

2.7 Field Type Inference from Excel

2.8 Field Suggestion Generator

2.9 GCP Vision Integration

3 System Architecture

4 Implementation Details

5 Improvements & Future Enhancements

1 · Project Overview

What is this project?

Sclera ADC AI Server is an intelligent Asset Data Collection platform built with FastAPI. It helps facilities and asset management teams automatically identify, classify, and validate building and industrial equipment assets — using a combination of computer vision, large language models, semantic search, and web intelligence.

What problem does it solve?

Managing thousands of physical assets (HVAC units, electrical panels, fire suppression systems, etc.) is slow and error-prone when done manually. Data entry teams must look up codes, classify equipment, and validate specifications by hand. This system automates that entire workflow using AI.

2 · AI Features

Feature 1: Asset Image Classification (Mobile)

Purpose

Takes a photo of a physical asset and automatically returns the most likely JACS classification codes for that asset.

Endpoint

```
POST /api/jacsid-mob
```

Workflow

1. Receive uploaded image(s)
2. Compress any image larger than 200 KB to reduce API cost and latency
3. Encode image(s) to Base64
4. Send to GPT-4o-mini with a vision prompt: 'Describe this asset in 50 words'
5. Receive a plain-English description of the asset
6. Convert that description into a vector (embedding) using text-embedding-3-large
7. Compare the embedding against the pre-loaded JACS code database using cosine similarity
8. Return the top matching JACS codes with confidence scores

Technologies Used

GPT-4o-mini (OpenAI) — vision understanding | text-embedding-3-large (OpenAI) — semantic vector search | NumPy — cosine similarity | Pillow — image compression

Edge Cases Handled

- No image provided → graceful empty result returned
- Very small/blurry images → model still attempts description
- Both asset + OCR images provided → combined for richer analysis; OCR preferred for factual details

Failure Handling

OpenAI API timeout → HTTP 500. Embedding creation failure → logged and request fails cleanly. Invalid image format → Pillow raises error before API call.

Feature 2: Asset Classification from Metadata (Mobile)

Purpose

Given a manufacturer name, model number, and description text, returns a ranked list of MasterFormat classification codes for the asset.

Endpoint

```
POST /api/classification-mobile-manual
```

Workflow

1. Receive manufacturer, model, and description
2. Build a web search query from the structured metadata
3. Call SerpAPI → Google Search returns top 10 organic results
4. Extract titles, snippets, and highlights from each result
5. Clean and concatenate all text into a rich context string
6. Create an embedding of that combined context using text-embedding-3-large
7. Compare against reference MasterFormat embeddings
8. Map the best-matching 6-digit MasterFormat code to its classification hierarchy (System → Asset → Sub-asset)
9. Return the top 5 ranked results

Technologies Used

SerpAPI — Google web search | text-embedding-3-large — semantic vector search | NumPy — cosine similarity | MasterFormat code hierarchy logic

Edge Cases Handled

- No search results returned → falls back to description-only embedding
- Manufacturer/model not found online → partial match still scored
- Description field missing → uses manufacturer + model alone

Failure Handling

SerpAPI failure → logged, request falls back gracefully. Empty search results → continues with available data.

Feature 3: Asset Classification from Image (Mobile)

Purpose

Classifies an asset by uploading its image and running a reverse image search to find similar products online, then maps results to MasterFormat codes.

Endpoint

```
POST /api/classification-mob-image
```

Workflow

1. Path A (OCR text provided): Normalize OCR text, send to GPT-4.1-mini to extract manufacturer/model/category
2. Path A: Build structured search query, run Google Search via SerpAPI, embed results and match against MasterFormat DB
3. Path B (Image provided): Compress image if over threshold
4. Path B: Send to SerpAPI Google Lens (reverse image search), retrieve visually similar product matches
5. Path B: Create embeddings from match descriptions, semantic search against MasterFormat database

Technologies Used

GPT-4.1-mini — text extraction from OCR output | SerpAPI Google Lens — reverse image search | text-embedding-3-large — semantic search | Pillow — image compression

Edge Cases Handled

- Both inputs provided → OCR text path takes priority
- Unreadable OCR text → GPT normalizes before use
- Google Lens returns no visual matches → empty result with proper status

Failure Handling

Lens API failure → logged, returns empty classification list. GPT extraction failure → raw OCR text used as fallback query.

Feature 4: JACS Code Prediction from Metadata (Mobile)

Purpose

Predicts JACS classification codes from text metadata (manufacturer, model, description) using a three-tier lookup strategy.

Endpoint

```
POST /api/mob-jacs-suggestion
```

Workflow

1. Receive asset list and validate required fields
2. Tier 1 — Direct Lookup: Check Excel database for exact manufacturer + model match
3. Tier 2 — Fuzzy Lookup: If no exact match, use RapidFuzz token sort similarity (manufacturer: 60%, model: 40%)
4. Tier 3 — Semantic Search: If no fuzzy match, send to Grok API to generate a 50-word description

5. Embed the description and search JACS database
6. Return best result with source label and confidence score

Technologies Used

RapidFuzz — fuzzy string matching | Grok-3 / Grok-4 (xAI) — asset description generation | text-embedding-3-large — semantic search | Pandas / Openpyxl — Excel database reading

Edge Cases Handled

- Missing manufacturer or model → skips direct lookup, goes straight to semantic
- Manufacturer alias names → Excel sheet includes alias columns
- Description field empty → Grok infers from manufacturer + model alone

Failure Handling

Grok API failure → retried up to 3 times. Excel file missing or corrupt → error logged, semantic path used. All tiers fail → item returned with null JACS ID and error flag.

Feature 5: Specification Validation

Purpose

Validates an asset's metadata fields against what is visible in uploaded photos and OCR images. Detects inconsistencies and missing information.

Endpoint

```
POST /api/spec-validation (internal)
```

Workflow

1. Receive metadata fields, asset image, and OCR image
2. Filter out date and ID fields (not validatable from images)
3. Encode images to Base64
4. Send all images + metadata to GPT-4o with a structured validation prompt
5. GPT cross-checks each metadata field against image evidence
6. Returns errors (definite mismatch), warnings (uncertain), and confidence scores per field

Technologies Used

GPT-4o — multimodal reasoning (vision + text) | OpenAI API (vision)

Edge Cases Handled

- Field not visible in image → returns 'cannot verify' rather than error
- No image provided → skips image-based checks
- Partially visible nameplate → GPT notes limited visibility

Failure Handling

GPT timeout → HTTP 500 returned to caller. Image too large → compressed before sending.

Feature 6: Column / Field Mapping

Purpose

Automatically maps column headers from a source dataset (e.g., a customer's spreadsheet) to a standard target schema using semantic understanding.

Endpoint

```
POST /api/ai-field-mapping/compare
```

Workflow

1. Receive source column names and target schema names

2. Build a structured prompt asking GPT-4o to semantically match each source column to the most appropriate target
3. GPT returns a JSON array of source → target pairs
4. Parse and return results

Technologies Used

GPT-4o — semantic column matching (Temperature: 1.0 for flexible reasoning)

Edge Cases Handled

- No clear match → GPT returns null for that column
- Multiple source columns for same target → GPT picks the best one
- Different naming conventions (camelCase, snake_case, abbreviations) → handled semantically

Failure Handling

Invalid JSON from GPT → retry or return error. GPT returns partial matches → incomplete map returned with available data.

Feature 7: Field Type Inference from Excel

Purpose

Analyzes an uploaded Excel file and infers the data type and possible values for each column, producing a structured field schema.

Endpoint

```
POST /api/asset-field-ai/import
```

Workflow

1. Accept Excel file upload
2. Read column names and sample unique values using Pandas
3. Send column names + sample values to Grok API
4. Grok infers field type: text, number, date, email, dropdown, or multiple_checkbox
5. For dropdown/checkbox types, unique values are extracted and returned as options
6. Return structured field metadata array

Technologies Used

Grok-3 / Grok-4 (xAI) — field type classification | Pandas / Openpyxl — Excel file reading

Edge Cases Handled

- Mixed data types in a column → Grok infers most likely type
- Very high cardinality columns → treated as 'text' not 'dropdown'
- Empty columns → type defaults to 'text'

Failure Handling

Grok API failure → retried up to 3 times. Corrupt Excel file → parsing error returned before API call.

Feature 8: Field Suggestion Generator

Purpose

Given an asset's metadata structure, suggests additional relevant fields the user might want to capture for that asset type.

Endpoint

```
POST /api/asset-field-ai/suggestion
```

Workflow

1. Receive asset type and list of existing fields
2. Send to GPT-4o with a prompt to suggest relevant additional fields

3. GPT returns suggested field names with types and example options
4. Return structured suggestions

Technologies Used

GPT-4o — domain knowledge for field suggestions | OpenAI API

Edge Cases Handled

- Unknown asset type → GPT provides generic equipment fields
- All relevant fields already present → GPT returns empty or minimal list

Failure Handling

GPT timeout → HTTP 500 returned to caller.

Feature 9: GCP Vision Integration

Purpose

Performs low-level computer vision tasks on uploaded images: OCR text extraction, logo detection, label detection, and object localization.

Endpoint

```
Internal — called by upstream features
```

Workflow

1. Encode images to Base64
2. Send both images concurrently to Google Cloud Vision API (async parallel)
3. Aggregate results: OCR text, detected logos, labels, and localized objects
4. Return combined output for downstream AI steps

Technologies Used

Google Cloud Vision API | asyncio — parallel image processing

Edge Cases Handled

- GCP API error → partial results returned where available
- Image unreadable → empty results for that image, other still processed

Failure Handling

GCP API error → partial results returned where available. Both images processed in parallel; one failure does not block the other.

3 · System Architecture

Architecture Overview

The system follows a layered API architecture with clear separation between:

- Routing layer — FastAPI routes that receive requests and delegate to services
- Service layer — Business logic combining multiple AI calls into coherent workflows
- AI integration layer — Thin wrappers around OpenAI, Grok, SerpAPI, and GCP
- Data layer — Pre-loaded JSONL embedding databases and Excel lookup files

All reference embeddings are loaded into memory at server startup, making similarity searches fast without disk I/O on each request. The server runs on port 8002 in a Docker container.

Service Layer Responsibilities

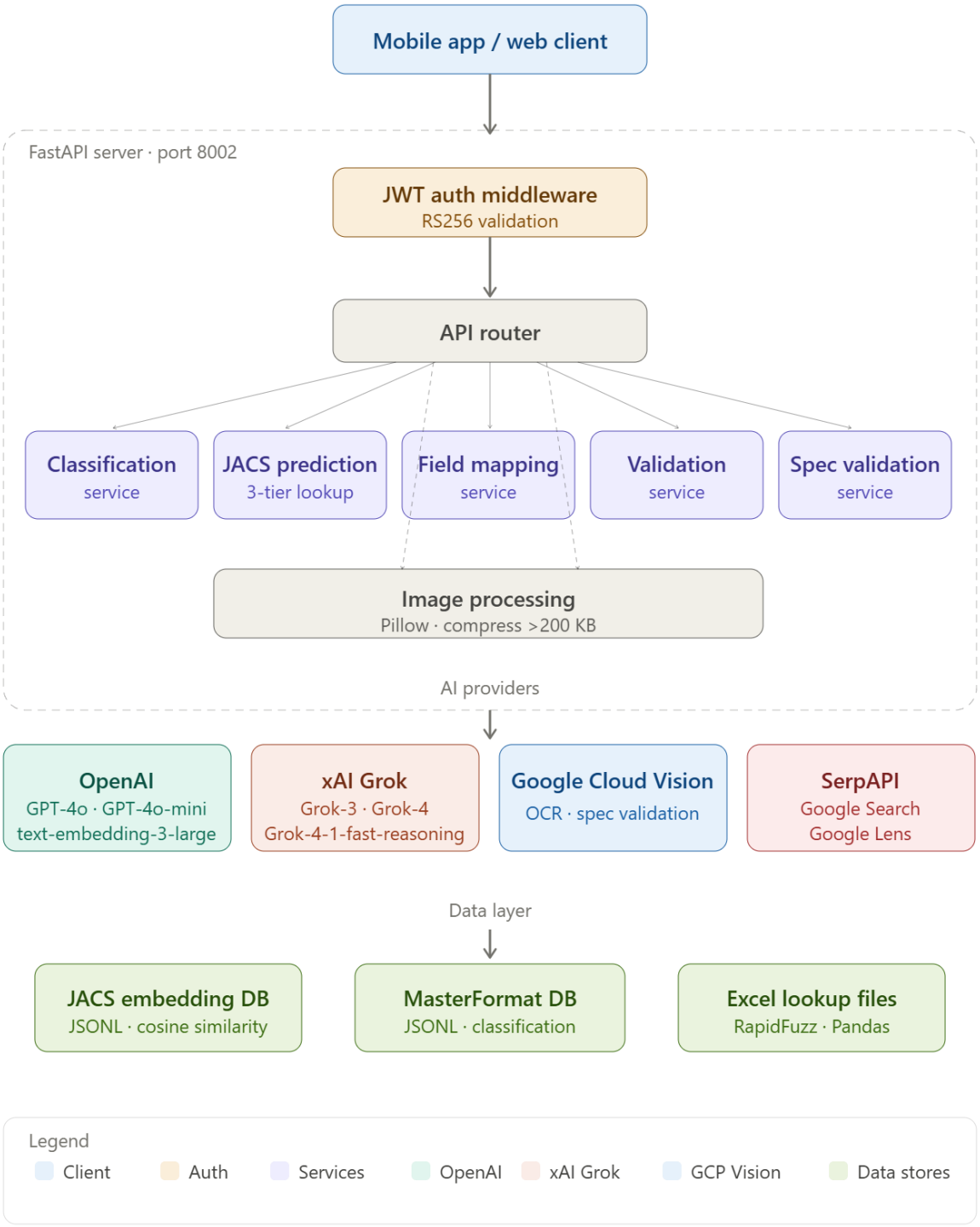
Service	Responsibilities	AI Used
Classification Service	MasterFormat code lookup via web + image search	SerpAPI, GPT-4.1-mini, Embeddings
JACS Prediction Service	Three-tier JACS code prediction	Grok-3/4, Embeddings, Excel
Field Mapping Service	Column mapping + field suggestions	GPT-4o
Validation Service	Fuzzy matching, Excel lookup	RapidFuzz, Grok
Spec Validation Service	Cross-validate metadata vs. images	GPT-4o Vision, GCP Vision

Embedding Databases (Loaded at Startup)

Database	File	Used By
JACS Codes	jacs_code_Embedding.jsonl	Features 1, 4
MasterFormat Codes	Embeddings_Master_code.jsonl	Features 2, 3
SubType Codes	SubType Embedding DB	Features 2, 3
Excel Lookup	Excel files (.xlsx)	Feature 4 — direct + fuzzy lookup

System Architecture Diagram

The diagram below shows the full system architecture — from the mobile/web client through JWT authentication, API routing, service layer, AI providers, and data stores.


Mobile app / web client
FastAPI server · port 8002
JWT auth middleware
RS256 validation
API router
Classification
service
JACS prediction
3-tier lookup
Field mapping
service
Validation
service
Spec validation
service
Image processing
Pillow · compress >200 KB
AI providers
OpenAI
GPT-4o · GPT-4o-mini
text-embedding-3-large
xAI Grok
Grok-3 · Grok-4
Grok-4-1-fast-reasoning
Google Cloud Vision
OCR · spec validation
SerpAPI
Google Search
Google Lens
Data layer
JACS embedding DB
JSONL · cosine similarity
MasterFormat DB
JSONL · classification
Excel lookup files
RapidFuzz · Pandas
Legend
Client
Auth
Services
OpenAI
xAI Grok
GCP Vision
Data stores

4 · Implementation Details

Key Design Decisions

Multi-tier Lookup Strategy

The JACS prediction system uses three tiers — exact Excel match, fuzzy match, then semantic search — ordered from cheapest to most expensive. This keeps costs low for well-known equipment while still handling unknown assets.

Pre-loaded Embeddings

All three embedding databases (JACS, MasterFormat, SubType) are loaded into memory at startup via the FastAPI lifespan event. This trades a small memory footprint for zero per-request disk I/O.

Multiple AI Providers

The system deliberately uses multiple providers (OpenAI, xAI Grok, Google Cloud Vision, SerpAPI) rather than a single one. Each provider is used where it performs best: GPT-4o for vision and field mapping, Grok for fast batch description generation, Google Lens for visual product identification, and SerpAPI for real-world web enrichment.

Temperature Tuning

Classification tasks use temperature 0–0.2 (deterministic, consistent outputs). Creative tasks such as field suggestions use temperature 0.7–1.0 (varied, broader suggestions).

Batch Processing with Graceful Fallback

Grok calls attempt 50 items per batch. On failure (rate limit or timeout), automatically retries with 25 items. This balances throughput with reliability.

Image Compression Threshold

Images over 200 KB are auto-compressed using Pillow before any API call. This reduces OpenAI vision API cost and latency without meaningful loss of classification accuracy.

MasterFormat Code Hierarchy Parsing

Classification codes follow a strict 6-digit structure parsed deterministically: Digits 1–2 → System Type (e.g., '23' → HVAC), Digits 1–4 → Asset Type, All 6 digits → Asset Sub-Type.

JWT Multi-tenant Authentication

Every route is protected by JWT middleware that validates RS256-signed tokens. Signing keys are fetched dynamically from the issuer's JWKS URI, supporting multi-tenant deployments without hardcoded secrets.

5 · Improvements & Future Enhancements

Performance Improvements

Improvement	Benefit
Async Grok Calls	Current Grok processing is sequential with 0.4s delay. Migrating to a true async batch queue would significantly increase throughput for large asset imports.
Embedding Caching	Cache embeddings for repeated manufacturer/model combinations. If 'Carrier 30XA-350' has been embedded before, skip the API call and reuse the stored vector.
Vector Database	Replace linear JSONL cosine search with a proper vector database (Pinecone, Qdrant, or pgvector). Scales from thousands to millions of records without performance degradation.

Accuracy Improvements

Improvement	Benefit
Fine-tuning	Fine-tune a smaller model specifically on historical asset-to-JACS-code mappings. Could replace the Grok → embedding pipeline for common asset types with higher accuracy and lower cost.
Feedback Loop	Allow users to correct AI predictions. Store corrections and incorporate them into the Excel lookup database, gradually reducing reliance on expensive LLM calls.
Confidence Thresholds	Implement configurable confidence thresholds. Results below a minimum score could trigger a 'needs review' flag rather than being automatically applied.

Scaling Ideas

Improvement	Benefit
Horizontal Scaling	The server is stateless (embeddings are read-only in memory). Multiple Docker replicas behind a load balancer would scale linearly.
Background Job Queue	For large Excel imports, move processing to a background queue (Celery + Redis) and return a job ID immediately. The client polls for results rather than waiting synchronously.
Rate Limit Management	Implement a central token bucket or leaky bucket for Grok/OpenAI API calls to prevent rate limit errors during traffic spikes.

New Feature Ideas

Improvement	Benefit
Document Ingestion	Accept PDF O&M; manuals and automatically extract specifications to pre-populate asset fields.
Continuous Learning	Periodically retrain or update embedding databases as new asset types enter the system.
Confidence Explanations	Return human-readable explanations alongside each classification (e.g., 'Matched because nameplate text mentions Carrier 30XA').
Multi-language Support	Extend OCR extraction and prompts to handle non-English nameplates (Spanish, French, Chinese) common in global deployments.